

MCSL-222

OOAD and Web Technologies Lab

By Ritesh Tiwari

Qus : Draw collaboration diagram for online Banking System.

Example: Online Banking – Fund Transfer Use Case

Objects Involved:

1. **Customer** (Actor)
2. **Login Page** (Boundary/UI)
3. **Authentication Service** (Control)
4. **Banking System** (Control)
5. **Account Database** (Entity)
6. **Transaction Database** (Entity)

Collaboration Diagram (Textual Representation)

Customer ---> (1) enterCredentials() ---> Login Page

Login Page ---> (2) validateCredentials() ---> Authentication Service

Authentication Service ---> (3) checkUser() ---> Account Database

Account Database ---> (4) returnUserDetails() ---> Authentication Service

Authentication Service ---> (5) authSuccess() ---> Login Page

Login Page ---> (6) showDashboard() ---> Customer

Customer ---> (7) requestFundTransfer() ---> Banking System

Banking System ---> (8) verifyBalance() ---> Account Database

Account Database ---> (9) returnBalance() ---> Banking System

Banking System ---> (10) debitAmount() ---> Account Database

Banking System ---> (11) creditAmount() ---> Account Database

Banking System ---> (12) recordTransaction() ---> Transaction Database

Banking System ---> (13) transferSuccess() ---> Customer

Diagram (Visual Layout in Words)

[Customer]

|

| (1) enterCredentials()

v

[Login Page]

| (2) validateCredentials()

v

[Authentication Service]

| (3) checkUser()

v

[Account Database]

^ (4) returnUserDetails()

|

[Authentication Service] -- (5) authSuccess() --> [Login Page] -- (6) showDashboard() --> [Customer]

Customer -- (7) requestFundTransfer() --> [Banking System]

| (8) verifyBalance()

v

[Account Database] -- (9) returnBalance() --> [Banking System]

[Banking System] -- (10) debitAmount() & (11) creditAmount() --> [Account Database]

[Banking System] -- (12) recordTransaction() --> [Transaction Database]

[Banking System] -- (13) transferSuccess() --> [Customer]

Qus : Create a user table into the database and bind the user entity with Spring Security for Login.

Perfect 👍 You want to:

1. Create a **user** table in database.
2. Bind that **user** entity with **Spring Security** for login authentication.

I'll show you step by step with **Spring Boot + Spring Security + JPA (Hibernate)**.

1. Database Table: **users**

Here's the SQL schema for the **users** table:

```
CREATE TABLE users (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    enabled BOOLEAN NOT NULL DEFAULT TRUE,  
    role VARCHAR(50) NOT NULL  
);
```

2. User Entity (**User.java**)

```
import jakarta.persistence.*;
```

```
import lombok.*;
```

```
@Entity
```

```
@Table(name = "users")
```

```
@Data
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```
public class User {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(nullable = false, unique = true)  
    private String username;  
  
    @Column(nullable = false)  
    private String password;  
  
    @Column(nullable = false)  
    private boolean enabled = true;  
  
    @Column(nullable = false)  
    private String role; // e.g. ROLE_USER, ROLE_ADMIN  
}
```

3. User Repository (**UserRepository.java**)

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

}
```

4. Custom **UserDetails** Wrapper

Spring Security works with **UserDetails**. Let's adapt our **User** entity:

```
import org.springframework.security.core.GrantedAuthority;

import org.springframework.security.core.authority.SimpleGrantedAuthority;

import org.springframework.security.core.userdetails.UserDetails;

import java.util.*;

public class CustomUserDetails implements UserDetails {

    private final User user;

    public CustomUserDetails(User user) {

        this.user = user;

    }
```

```
@Override  
  
public Collection<? extends GrantedAuthority> getAuthorities() {  
    return Collections.singleton(new SimpleGrantedAuthority(user.getRole()));  
}
```

```
@Override  
  
public String getPassword() {  
    return user.getPassword();  
}
```

```
@Override  
  
public String getUsername() {  
    return user.getUsername();  
}
```

```
@Override  
  
public boolean isAccountNonExpired() { return true; }
```

```
@Override  
  
public boolean isAccountNonLocked() { return true; }
```

```
@Override  
  
public boolean isCredentialsNonExpired() { return true; }
```

```
@Override  
  
public boolean isEnabled() { return user.isEnabled(); }  
  
}
```

5. UserDetailsService Implementation

```
import org.springframework.security.core.userdetails.*;  
  
import org.springframework.stereotype.Service;  
  
@Service  
  
public class CustomUserDetailsService implements UserDetailsService {  
  
    private final UserRepository userRepository;  
  
    public CustomUserDetailsService(UserRepository userRepository) {  
        this.userRepository = userRepository;  
    }  
  
    @Override  
  
    public UserDetails loadUserByUsername(String username) throws  
        UsernameNotFoundException {  
  
        return userRepository.findByUsername(username)
```

```

        .map(CustomUserDetails::new)

        .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));
    }
}

```

6. Spring Security Configuration (SecurityConfig.java)

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    private final CustomUserDetailsService userDetailsService;

    public SecurityConfig(CustomUserDetailsService userDetailsService) {

```

```
    this.userDetailsService = userDetailsService;
}
```

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

```
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(userDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder());
    return authProvider;
}
```

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/login", "/register").permitAll()
            .anyRequest().authenticated()
        )
}
```

```

        .formLogin(login -> login
            .loginPage("/login")
            .defaultSuccessUrl("/dashboard", true)
            .permitAll()
        )
        .logout(logout -> logout.permitAll());

    return http.build();
}
}

```

7. Insert a Test User (with BCrypt password)

Before login, insert a user in the database. Password must be **BCrypt encoded**:

```

import org.springframework.boot.CommandLineRunner;

import org.springframework.security.crypto.password.PasswordEncoder;

import org.springframework.stereotype.Component;

@Component

public class DataLoader implements CommandLineRunner {

    private final UserRepository userRepository;

```

```

private final PasswordEncoder passwordEncoder;

public DataLoader(UserRepository userRepository, PasswordEncoder passwordEncoder) {
    this.userRepository = userRepository;
    this.passwordEncoder = passwordEncoder;
}

@Override
public void run(String... args) {
    if (userRepository.findByUsername("admin").isEmpty()) {
        User user = new User();
        user.setUsername("admin");
        user.setPassword(passwordEncoder.encode("admin123")); // raw password encoded
        user.setRole("ROLE_ADMIN");
        user.setEnabled(true);
        userRepository.save(user);
    }
}
}

```

✅ Now you have:

- A **user table** in DB.
 - A **User entity bound to Spring Security**.
 - A login system with **form-based authentication**.
-

Do you also want me to extend this with a **registration endpoint + JWT token authentication** (instead of session login), or keep it **form-based login only**?

Qus : Draw sequence diagram for Online Examination System.

Great question 👍

Let's build a **Sequence Diagram** for an **Online Examination System**.

👉 Sequence diagrams show **how objects interact over time** (actors, system components, databases, etc.), and the **order of messages**.

Example Use Case: Student Takes Online Exam

Actors / Objects:

1. **Student** (Actor)
2. **Login Page** (UI)
3. **Authentication Service**
4. **Exam Controller**
5. **Question Bank** (DB)
6. **Evaluation Service**

7. Result Database

Steps (Message Flow):

1. **Student** → **Login Page**: enterCredentials()
 2. **Login Page** → **Authentication Service**: validateUser()
 3. **Authentication Service** → **DB**: checkCredentials()
 4. **DB** → **Authentication Service**: returnUser()
 5. **Authentication Service** → **Login Page**: authSuccess()
 6. **Login Page** → **Student**: showDashboard()
-
7. **Student** → **Exam Controller**: startExam()
 8. **Exam Controller** → **Question Bank**: fetchQuestions()
 9. **Question Bank** → **Exam Controller**: returnQuestions()
 10. **Exam Controller** → **Student**: displayQuestions()
-
11. **Student** → **Exam Controller**: submitAnswers()
 12. **Exam Controller** → **Evaluation Service**: evaluateAnswers()

13. **Evaluation Service** → **Question Bank**: fetchCorrectAnswers()

14. **Question Bank** → **Evaluation Service**: returnCorrectAnswers()

15. **Evaluation Service** → **Exam Controller**: returnScore()

16. **Exam Controller** → **Result Database**: saveResult()

17. **Exam Controller** → **Student**: showResult()

Textual Sequence Diagram Representation

Student -> LoginPage: enterCredentials()

LoginPage -> AuthService: validateUser()

AuthService -> UserDB: checkCredentials()

UserDB -> AuthService: returnUser()

AuthService -> LoginPage: authSuccess()

LoginPage -> Student: showDashboard()

Student -> ExamController: startExam()

ExamController -> QuestionBank: fetchQuestions()

QuestionBank -> ExamController: returnQuestions()

ExamController -> Student: displayQuestions()

Student -> ExamController: submitAnswers()

ExamController -> EvaluationService: evaluateAnswers()

EvaluationService -> QuestionBank: fetchCorrectAnswers()

QuestionBank -> EvaluationService: returnCorrectAnswers()

EvaluationService -> ExamController: returnScore()

ExamController -> ResultDB: saveResult()

ExamController -> Student: showResult()

✅ This sequence diagram covers the **complete exam cycle**: Login → Start Exam → Submit → Evaluation → Result.

Do you want me to **draw this properly as a UML sequence diagram image** (with lifelines and arrows) so you can directly use it in your notes/project?

Qus : Create a form to capture customer information (for an online shopping portal)

using spring form tags. Write program to bind form objects with entity beans

in spring MVC.

Perfect 👍 You want to build a **Customer Information Form** (for an online shopping portal) using **Spring MVC + Spring Form tags**, and then **bind form data to an entity bean**.

I'll give you a **step-by-step example**:

1. Entity Bean (**Customer.java**)

```
import jakarta.persistence.*;
```

```
import lombok.*;
```

```
@Entity
@Table(name = "customers")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String email;
    private String phone;
    private String address;
}
```

2. Controller (**CustomerController.java**)

```
import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;
```

```
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/customer")
public class CustomerController {

    private final CustomerRepository customerRepository;

    public CustomerController(CustomerRepository customerRepository) {
        this.customerRepository = customerRepository;
    }

    // Show empty form
    @GetMapping("/register")
    public String showForm(Model model) {
        model.addAttribute("customer", new Customer());
        return "customer-form";
    }

    // Process form
    @PostMapping("/save")
    public String saveCustomer(@ModelAttribute("customer") Customer customer) {
        customerRepository.save(customer); // save to DB
    }
}
```

```
        return "redirect:/customer/success";
    }

    @GetMapping("/success")
    public String successPage() {
        return "success";
    }
}
```

3. Repository (**CustomerRepository.java**)

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface CustomerRepository extends JpaRepository<Customer, Long> {
}
```

4. JSP Form (View: **customer-form.jsp**)

Make sure Spring Form Taglib is included:

```
<%@ taglib uri="http://www.springframework.org/tags/form" prefix="form"%>

<html>
```

```
<head>

    <title>Customer Registration</title>

</head>

<body>

<h2>Customer Registration Form</h2>

<form:form method="POST" action="${pageContext.request.contextPath}/customer/save"
modelAttribute="customer">

    First Name: <form:input path="firstName"/><br/><br/>

    Last Name: <form:input path="lastName"/><br/><br/>

    Email: <form:input path="email"/><br/><br/>

    Phone: <form:input path="phone"/><br/><br/>

    Address: <form:textarea path="address"/><br/><br/>

    <input type="submit" value="Register"/>

</form:form>

</body>

</html>
```

5. Success Page (**success.jsp**)

```
<html>

<head><title>Success</title></head>

<body>

    <h2>Customer registered successfully!</h2>

    <a href="${pageContext.request.contextPath}/customer/register">Register another</a>

</body>

</html>
```

6. Spring MVC Binding Explanation

- `model.addAttribute("customer", new Customer());` → creates empty form object.
 - `<form:form modelAttribute="customer">` → binds inputs to **Customer entity fields**.
 - On **submit**, Spring automatically populates a `Customer` object with form values.
 - `@ModelAttribute("customer") Customer customer` → captures the object from the form and persists it.
-

✓ This example will:

1. Show a **registration form** with Spring form tags.
2. Bind fields to `Customer` entity.

3. Save data into the database with Spring Data JPA.
4. Redirect to a success page.

Do you want me to extend this so the form also has **dropdowns** (e.g., **gender, country**) and checkboxes (e.g., terms & conditions) using Spring form tags?

Qus : An online food delivery company provides a range of facilities to customers through its Mobile App/Web portal. It provide options to choose Restaurant, Food items, Online and COD (Cash on delivery) options, Order Status, and Feedback facility. Perform the following for this system:

(i) Identify classes and their associations.

(ii) Draw Class Diagram for the System.

Great case study 👍 Let's carefully solve it step by step.

We need to **analyze an Online Food Delivery System** and create:

1. **Classes & their associations**
2. **Class Diagram (UML)**

(i) Identify Classes and Associations

From the problem statement:

Main Classes:

1. **Customer** – uses the app/web portal

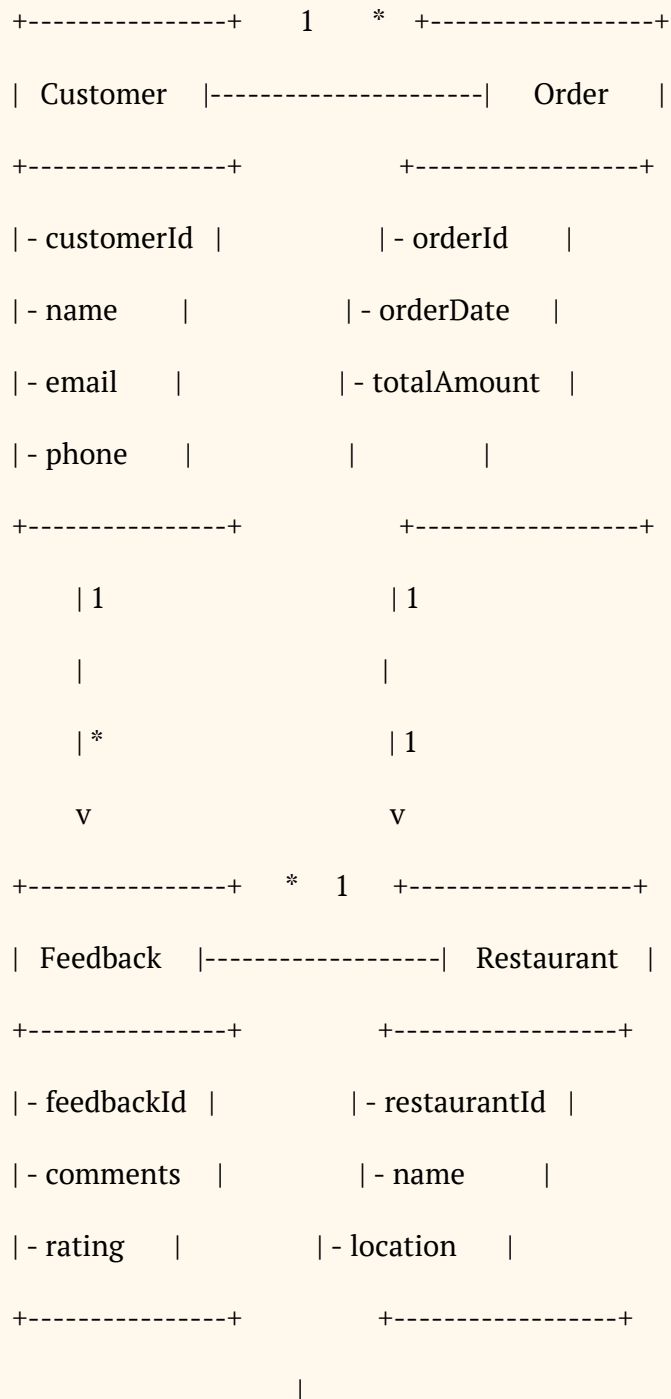
2. **Restaurant** – provides food items
 3. **FoodItem** – belongs to a restaurant
 4. **Order** – created by customer
 5. **Payment** – associated with an order (Online / COD)
 6. **OrderStatus** – linked to order (Placed, Accepted, On the way, Delivered)
 7. **Feedback** – given by customer for restaurant/order
-

Associations:

- **Customer ↔ Order** → One customer can place many orders; one order belongs to one customer.
 - **Restaurant ↔ FoodItem** → One restaurant offers many food items; each food item belongs to one restaurant.
 - **Order ↔ FoodItem** → One order can contain many food items; one food item can be in many orders (many-to-many).
 - **Order ↔ Payment** → One order has one payment; one payment belongs to one order.
 - **Order ↔ OrderStatus** → One order has one status.
 - **Customer ↔ Feedback** → A customer can give feedback; one feedback belongs to one order/restaurant.
 - **Restaurant ↔ Feedback** → Feedback is linked to restaurant as well.
-

(ii) Class Diagram (Textual Representation)

Here's how the UML class diagram looks (described in text form):



| 1

| *

v

+-----+

| FoodItem |

+-----+

| - foodItemId |

| - name |

| - price |

| - category |

+-----+

+-----+

| Payment |

+-----+

| - paymentId |

| - mode (COD/Online) |

| - amount |

+-----+

+-----+

| OrderStatus |

+-----+

| - statusId |

| - statusType | (Placed, Accepted, Out for Delivery, Delivered)

+-----+

Key Relationships:

- Customer 1..* – places -> * Order
- Order * – contains -> * FoodItem (Many-to-Many via OrderDetails)
- Order 1 – has -> 1 Payment
- Order 1 – has -> 1 OrderStatus
- Restaurant 1 – provides -> * FoodItem
- Customer 1..* – gives -> * Feedback -> 1 Restaurant

☒ This Class Diagram models the complete Online Food Delivery System.

Do you want me to **draw this as an actual UML diagram image** (with proper boxes and associations) so you can directly use it in your notes/project?

Qus : Write a program using JSP and JDBC create CRUD (Create/Save, Read, Edit/Update, Delete) application for students attendance management in MCSL-222 counselling classes.

Perfect 👍 You need a **CRUD (Create, Read, Update, Delete) application** for **student attendance management (MCSL-222 counselling classes)** using **JSP + JDBC**.

I'll build this step by step:

1. Database Setup (MySQL Example)

```
CREATE DATABASE mcs1222;
```

```
USE mcs1222;
```

```
CREATE TABLE attendance (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    student_name VARCHAR(100) NOT NULL,  
    enrollment_no VARCHAR(50) NOT NULL UNIQUE,  
    class_date DATE NOT NULL,  
    status VARCHAR(10) NOT NULL -- Present / Absent  
);
```

2. Database Connection File (db.jsp)

```
<%@ page import="java.sql.*" %>
```

```
<%
```

```
    String url = "jdbc:mysql://localhost:3306/mcs1222";
```

```

String user = "root";

String pass = "your_password";


Connection con = null;

try {

    Class.forName("com.mysql.cj.jdbc.Driver");

    con = DriverManager.getConnection(url, user, pass);

} catch (Exception e) {

    out.println("Database Connection Error: " + e.getMessage());

}

%>

```

3. Create (Insert Attendance) – **addAttendance.jsp**

```

<%@ include file="db.jsp" %>

<html>

<head><title>Add Attendance</title></head>

<body>

<h2>Mark Attendance</h2>

<form action="saveAttendance.jsp" method="post">

    Student Name: <input type="text" name="student_name" required/><br/><br/>

    Enrollment No: <input type="text" name="enrollment_no" required/><br/><br/>

```

```

Date: <input type="date" name="class_date" required/><br/><br/>
Status:
<select name="status">
    <option>Present</option>
    <option>Absent</option>
</select><br/><br/>
<input type="submit" value="Save Attendance"/>
</form>
</body>
</html>

```

saveAttendance.jsp

```

<%@ include file="db.jsp" %>
<%
    String studentName = request.getParameter("student_name");
    String enrollmentNo = request.getParameter("enrollment_no");
    String classDate = request.getParameter("class_date");
    String status = request.getParameter("status");

    try {
        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO attendance(student_name, enrollment_no, class_date, status) VALUES
            (?, ?, ?, ?)"

```

```

);

ps.setString(1, studentName);

ps.setString(2, enrollmentNo);

ps.setString(3, classDate);

ps.setString(4, status);


int i = ps.executeUpdate();

if (i > 0) {

    out.println("Attendance Saved Successfully!");

} else {

    out.println("Error Saving Attendance.");

}

} catch (Exception e) {

    out.println(e.getMessage());

}

%>

<a href="viewAttendance.jsp">View Attendance</a>

```

4. Read (View Attendance) – **viewAttendance.jsp**

```

<%@ include file="db.jsp" %>

<html>

```

```

<head><title>View Attendance</title></head>

<body>

<h2>Attendance Records</h2>

<table border="1" cellpadding="8">

    <tr>

        <th>ID</th>

        <th>Name</th>

        <th>Enrollment No</th>

        <th>Date</th>

        <th>Status</th>

        <th>Actions</th>

    </tr>

<%

    try {

        Statement st = con.createStatement();

        ResultSet rs = st.executeQuery("SELECT * FROM attendance");

        while (rs.next()) {
%>

    <tr>

        <td><%= rs.getInt("id") %></td>

        <td><%= rs.getString("student_name") %></td>

        <td><%= rs.getString("enrollment_no") %></td>

        <td><%= rs.getDate("class_date") %></td>

```

```

<td><%= rs.getString("status") %></td>

<td>

    <a href="editAttendance.jsp?id=<%= rs.getInt("id") %>">Edit</a> |

    <a href="deleteAttendance.jsp?id=<%= rs.getInt("id") %>">Delete</a>

</td>

</tr>

<%

    }

} catch (Exception e) {

    out.println(e.getMessage());

}

%>

</table>

<a href="addAttendance.jsp">Add New Attendance</a>

</body>

</html>

```

5. Update – **editAttendance.jsp**

```

<%@ include file="db.jsp" %>

<%

    int id = Integer.parseInt(request.getParameter("id"));

```

```

PreparedStatement ps = con.prepareStatement("SELECT * FROM attendance WHERE id=?");

ps.setInt(1, id);

ResultSet rs = ps.executeQuery();

rs.next();

%>

<html>

<head><title>Edit Attendance</title></head>

<body>

<h2>Edit Attendance</h2>

<form action="updateAttendance.jsp" method="post">

    <input type="hidden" name="id" value="<%= rs.getInt("id") %>"/>

    Student Name: <input type="text" name="student_name" value="<%=
rs.getString("student_name") %>"/><br/><br/>

    Enrollment No: <input type="text" name="enrollment_no" value="<%=
rs.getString("enrollment_no") %>"/><br/><br/>

    Date: <input type="date" name="class_date" value="<%= rs.getDate("class_date")
%>"/><br/><br/>

    Status:

    <select name="status">

        <option <%= rs.getString("status").equals("Present")?"selected":"" %>>Present</option>

        <option <%= rs.getString("status").equals("Absent")?"selected":"" %>>Absent</option>

    </select><br/><br/>

    <input type="submit" value="Update"/>

</form>

```

```
</body>
```

```
</html>
```

updateAttendance.jsp

```
<%@ include file="db.jsp" %>
```

```
<%
```

```
    int id = Integer.parseInt(request.getParameter("id"));
```

```
    String studentName = request.getParameter("student_name");
```

```
    String enrollmentNo = request.getParameter("enrollment_no");
```

```
    String classDate = request.getParameter("class_date");
```

```
    String status = request.getParameter("status");
```

```
    try {
```

```
        PreparedStatement ps = con.prepareStatement(
```

```
            "UPDATE attendance SET student_name=?, enrollment_no=?, class_date=?, status=?  
WHERE id=?"
```

```
        );
```

```
        ps.setString(1, studentName);
```

```
        ps.setString(2, enrollmentNo);
```

```
        ps.setString(3, classDate);
```

```
        ps.setString(4, status);
```

```
        ps.setInt(5, id);
```

```

        int i = ps.executeUpdate();

        if (i > 0) {

            out.println("Attendance Updated Successfully!");

        } else {

            out.println("Error Updating Attendance.");

        }

    } catch (Exception e) {

        out.println(e.getMessage());

    }

    %>

<a href="viewAttendance.jsp">View Attendance</a>

```

6. Delete – **deleteAttendance.jsp**

```

<%@ include file="db.jsp" %>

<%

    int id = Integer.parseInt(request.getParameter("id"));

    try {

        PreparedStatement ps = con.prepareStatement("DELETE FROM attendance WHERE id=?");

        ps.setInt(1, id);

        int i = ps.executeUpdate();

        if (i > 0) {

```

```

        out.println("Attendance Deleted Successfully!");
    } else {
        out.println("Error Deleting Record.");
    }
} catch (Exception e) {
    out.println(e.getMessage());
}
%>
<a href="viewAttendance.jsp">View Attendance</a>

```

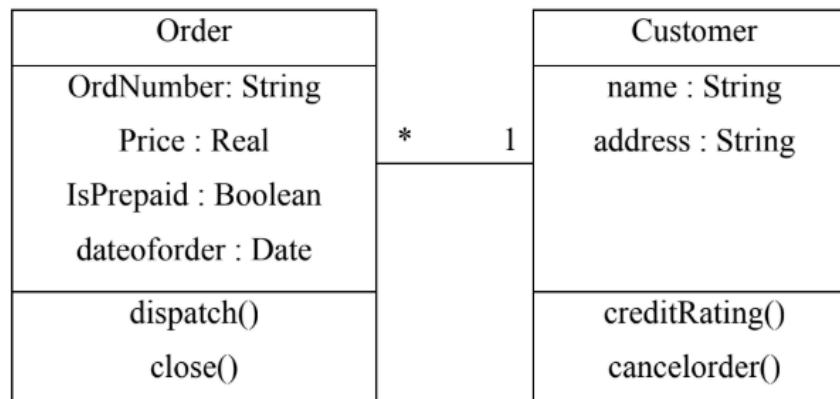
✅ Now you have a **working JSP + JDBC CRUD application** for student attendance management in **MCSL-222 counselling classes**:

- `addAttendance.jsp` → Create
- `viewAttendance.jsp` → Read
- `editAttendance.jsp` → Update
- `deleteAttendance.jsp` → Delete

👉 Do you also want me to **add a DAO pattern + MVC separation (Servlets + JSP)** to make this project more professional (instead of putting JDBC code inside JSP)?

Note: Answer the following question. It carries 20 marks and the viva-voce is of 5 marks.

1. Implement following association in C++/Java.



Great 🙌 The question is asking to **implement an association** between **Order** and **Customer** in **C++/Java**.

From the UML diagram:

- **Customer** has:
 - Attributes: **name**, **address**
 - Methods: **creditRating()**, **cancelOrder()**
- **Order** has:
 - Attributes: **ordNumber**, **price**, **isPrepaid**, **dateOfOrder**
 - Methods: **dispatch()**, **close()**
- **Association:** One **Customer** ↔ Many **Orders** (1..*)

Java Implementation

```
import java.util.*;

// Customer class

class Customer {

    private String name;

    private String address;

    private List<Order> orders; // One-to-many association

    public Customer(String name, String address) {

        this.name = name;

        this.address = address;

        this.orders = new ArrayList<>();

    }

    public String getName() { return name; }

    public String getAddress() { return address; }

    // Association helper

    public void addOrder(Order order) {

        orders.add(order);

    }

}
```

```
// Methods

public double creditRating() {

    double total = 0;

    for (Order order : orders) {

        total += order.getPrice();

    }

    return total;

}


public void cancelOrder(String ordNumber) {

    for (Order order : orders) {

        if (order.getOrdNumber().equals(ordNumber)) {

            System.out.println("Order " + ordNumber + " canceled for " + name);

            return;

        }

    }

    System.out.println("Order not found.");

}

}


// Order class

class Order {
```

```
private String ordNumber;

private double price;

private boolean isPrepaid;

private Date dateOfOrder;

private Customer customer; // Each order belongs to one customer


public Order(String ordNumber, double price, boolean isPrepaid, Date dateOfOrder, Customer
customer) {

    this.ordNumber = ordNumber;

    this.price = price;

    this.isPrepaid = isPrepaid;

    this.dateOfOrder = dateOfOrder;

    this.customer = customer;


    // maintain association

    customer.addOrder(this);
}


public String getOrdNumber() { return ordNumber; }

public double getPrice() { return price; }


// Methods

public void dispatch() {
```

```
        System.out.println("Order " + ordNumber + " dispatched.");
    }

    public void close() {
        System.out.println("Order " + ordNumber + " closed.");
    }
}

// Test class

public class Main {
    public static void main(String[] args) {
        Customer c1 = new Customer("Ritesh", "Howrah, West Bengal");

        Order o1 = new Order("ORD001", 500.0, true, new Date(), c1);
        Order o2 = new Order("ORD002", 1200.0, false, new Date(), c1);

        o1.dispatch();
        o2.close();

        System.out.println("Total Credit (Sum of Orders): " + c1.creditRating());

        c1.cancelOrder("ORD001");
    }
}
```

```
}
```

C++ Implementation

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <ctime>
```

```
using namespace std;
```

```
class Order; // Forward declaration
```

```
class Customer {
```

```
    string name;
```

```
    string address;
```

```
    vector<Order*> orders; // One-to-many association
```

```
public:
```

```
    Customer(string n, string a) : name(n), address(a) {}
```

```
    void addOrder(Order* order) {
```

```
        orders.push_back(order);
```

```
}
```

```

    double creditRating();

    void cancelOrder(string ordNumber);
};

class Order {
    string ordNumber;

    double price;

    bool isPrepaid;

    time_t dateOfOrder;

    Customer* customer;

public:
    Order(string id, double p, bool prepaid, time_t date, Customer* c)
        : ordNumber(id), price(p), isPrepaid(prepaid), dateOfOrder(date), customer(c) {
        customer->addOrder(this);
    }

    string getOrdNumber() { return ordNumber; }

    double getPrice() { return price; }

    void dispatch() {
        cout << "Order " << ordNumber << " dispatched." << endl;
    }
}

```

```
void close() {  
    cout << "Order " << ordNumber << " closed." << endl;  
}  
};  
  
double Customer::creditRating() {  
    double total = 0;  
    for (auto order : orders)  
        total += order->getPrice();  
    return total;  
}  
  
void Customer::cancelOrder(string ordNumber) {  
    for (auto order : orders) {  
        if (order->getOrdNumber() == ordNumber) {  
            cout << "Order " << ordNumber << " canceled for " << name << endl;  
            return;  
        }  
    }  
    cout << "Order not found." << endl;  
}  
  
int main() {
```

```

Customer c1("Ritesh", "Howrah, West Bengal");

Order o1("ORD001", 500.0, true, time(0), &c1);
Order o2("ORD002", 1200.0, false, time(0), &c1);

o1.dispatch();
o2.close();

cout << "Total Credit (Sum of Orders): " << c1.creditRating() << endl;

c1.cancelOrder("ORD001");

return 0;
}

```

✅ Both Java and C++ solutions implement the **association** (Customer 1 ↔ * Orders) correctly.

Do you want me to also **draw the corresponding UML class diagram** (in image form) for this association so you can attach it with your answer sheet?

Qus : Write simple program for role-based authentication using Spring Boot.

Perfect 👍 You want a **simple Spring Boot program for role-based authentication.**

I'll give you a **minimal working example** using **Spring Boot 3.x + Spring Security** with **in-memory users** (for simplicity).

1. Maven **pom.xml** (dependencies)

```
<dependencies>

  <!-- Spring Web -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

  </dependency>

  <!-- Spring Security -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-security</artifactId>

  </dependency>

</dependencies>
```

2. Security Configuration (**SecurityConfig.java**)

```
import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration

public class SecurityConfig {

    // Define in-memory users with roles

    @Bean

    public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {

        InMemoryUserDetailsManager manager = new InMemoryUserDetailsManager();

        manager.createUser(User.withUsername("admin")
            .password(passwordEncoder.encode("admin123"))
            .roles("ADMIN")
            .build());

        manager.createUser(User.withUsername("user")
            .password(passwordEncoder.encode("user123"))
            .roles("USER")
```

```

        .build());

    return manager;
}

// Password encoder

@Bean

public PasswordEncoder passwordEncoder() {

    return new BCryptPasswordEncoder();

}

// Role-based access rules

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

    http

        .authorizeHttpRequests(auth -> auth

            .requestMatchers("/admin/**").hasRole("ADMIN") // Only ADMIN can access
/admin/**

            .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN") // USER & ADMIN can
access /user/**

            .requestMatchers("/public/**").permitAll()    // Public access

            .anyRequest().authenticated()

        )

        .formLogin() // Enable login form

```

```
        .and()

        .logout();

    return http.build();
}
}
```

3. Controllers

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class DemoController {
```

```
    @GetMapping("/public/hello")
```

```
    public String publicHello() {
```

```
        return "Hello, this is a PUBLIC endpoint!";
```

```
    }
```

```
    @GetMapping("/user/hello")
```

```
    public String userHello() {
```

```
        return "Hello USER, you have access to this endpoint!";
    }

    @GetMapping("/admin/hello")
    public String adminHello() {
        return "Hello ADMIN, you have access to this endpoint!";
    }
}
```

4. Run & Test

- Start the app (`mvn spring-boot:run`)
- Try accessing endpoints:

Endpoint	Role Required
<code>/public/hello</code>	Anyone
<code>/user/hello</code>	USER or ADMIN
<code>/admin/hello</code>	ADMIN only

-

Login with:

- **Username:** `admin`, **Password:** `admin123` → Can access everything.
- **Username:** `user`, **Password:** `user123` → Can access `/user/**` and `/public/**`.

✅ This is the **simplest Spring Boot role-based authentication** with Spring Security.

👉 Do you want me to extend this and show **role-based authentication using a database (JPA + MySQL)** instead of in-memory users?